DevIA Radar

Como desenvolvedores discutem Inteligência Artificial no desenvolvimento de software

Etapa 1 · PLN com expressões regulares e spaCy · coleta por Web API

Augusto Blois · Pedro Flávio Nogueira Ribeiro de Luna
Dados Não Estruturados · UFPB · 2026

1.180
issues coletadas do GitHub

633
repositórios diferentes

356.966
tokens processados

O que este trabalho investiga

Como desenvolvedores discutem o uso de IA no desenvolvimento de software?

Por que issues do GitHub

- Issue é onde o desenvolvedor descreve um problema real com as próprias palavras, sem filtro de marketing.
- A Search API é pública, gratuita e devolve JSON, o que o enunciado aceita como coleta por Web API.
- Texto humano, escrito por quem usa a ferramenta no trabalho, e não resposta de pesquisa fechada.

979
issues analisadas

29,6%
de ruído removido

13.391
lemas únicos



Vocabulário dominante do corpus, depois da limpeza por regex e da filtragem do spaCy.

code e review lideram o corpus. O segundo lugar de review é o primeiro indício de que a revisão de código é o ponto de atrito do uso de IA.

Teoria e parâmetros

Três técnicas, aplicadas em sequência sobre o mesmo corpus

1 Expressão regular

- Linguagem formal para descrever a FORMA de um texto.
- \b fronteira de palavra \d dígito
- + uma ou mais repetições
  *? preguiçoso
- Serve para duas coisas: limpar o ruído e extrair informação estruturada.

2 spaCy

- tokenizer → tagger → lemmatizer → parser → ner
- Token: unidade do texto. Lema: forma de dicionário.
- POS: classe gramatical. NER: entidade nomeada.
- Filtrar stopword é o que faz o ranking virar tema, e não gramática.

3 VADER

- Sentimento por léxico, sem treino nem dado rotulado.
- Entende negação, maiúscula e pontuação repetida.
- compound ≥ +0,05 → positivo
- compound ≤ −0,05 → negativo

Parâmetros do projeto

6 termos de busca · até 200 issues por termo · corte de 3.000 caracteres para o spaCy · corte de 2.000 para o VADER · mínimo de 5 menções para uma ferramenta entrar na média

Implementação 1: expressões regulares

Arquivo limpeza.py: a regex limpa o ruído e extrai informação na mesma passada

```
RE_BLOCO_CODIGO = re.compile(r"```[\s\S]*?```")
RE_VERSAO = re.compile(
    r"\bv?(\d{1,3}\.\d{1,3}(?:\.\d{1,4})?)\b")

# ordena do mais longo para o mais curto:
# senão 'copilot' casa antes de 'github copilot'
ordenados = sorted(termos, key=len, reverse=True)
```

[\s\S] casa qualquer caractere, inclusive quebra de linha, coisa que o ponto sozinho não faz. O *? é preguiçoso: para no primeiro fechamento de bloco em vez de engolir o arquivo inteiro.

Onde aprender

regex101.com explica cada pedaço do padrão enquanto você digita

ANTES

````
Bug with Copilot suggestions
@octocat reported in #1234 that Copilot
v1.2.3 breaks. See https://github.com/...
```python
def quebrado(): raise ValueError()
```
````

DEPOIS

```
Bug with Copilot suggestions reported that Copilot
breaks.
```

EXTRAÍDO PELA MESMA REGEX

ferramenta: GitHub Copilot · versão: 1.2.3

referência: #1234 · tem_codigo: True

Implementação 2: spaCy

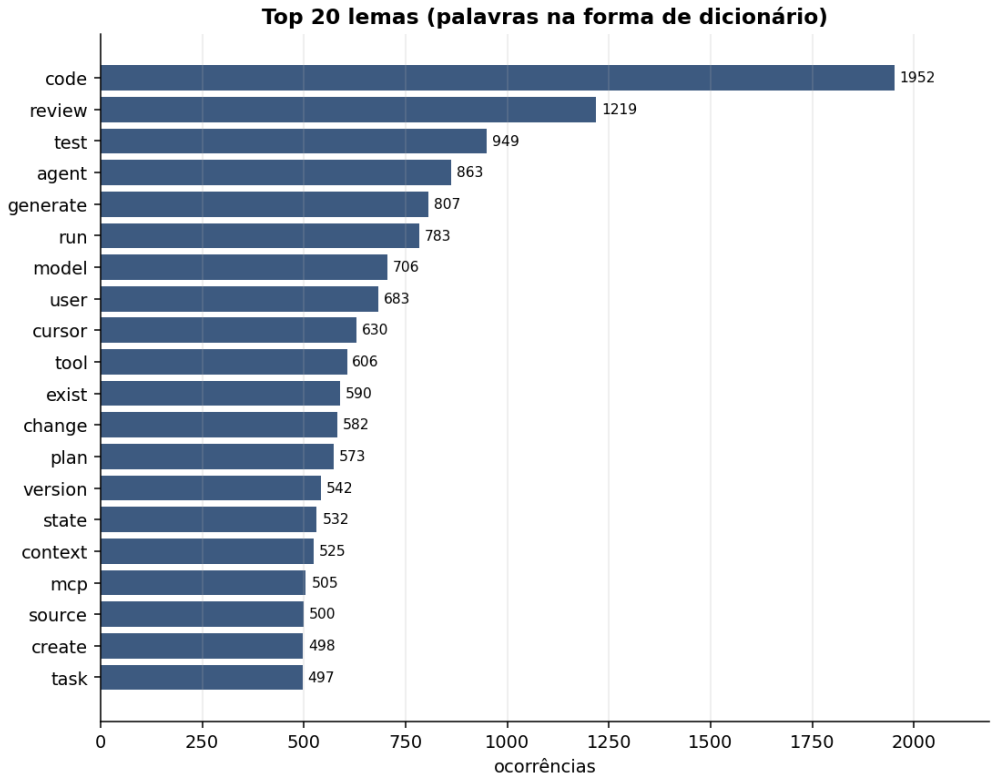
Arquivo pln.py: cada palavra volta anotada com classe gramatical, lema e entidade

`tokenizer` → `tagger` → `lemmatizer` → `parser` → **`ner`**

```
for doc in nlp.pipe(textos, batch_size=50):
    uteis = [t for t in doc if _token_util(t)]
    freq_lemas.update(
        t.lemma_.lower() for t in uteis)
```

- nlp.pipe processa em lote, bem mais rápido que chamar nlp(texto) dentro de um laço.
- Contar lema em vez de palavra crua: running, ran e runs viram a mesma entrada, e o ranking para de ficar picado.
- O filtro corta stopword, pontuação, número e palavra com menos de 3 letras.

Onde aprender: course.spacy.io, curso oficial gratuito e interativo



Diário de bordo

Quatro erros que apareceram na implementação, e o que cada um exigiu

1. **Bots de CI dominavam o corpus e invertiam a conclusão**

   Filtro por autor [bot] e por título. 198 issues fora, e as médias viraram positivas

2. **spaCy lia ### como valor monetário (tipo MONEY)**

   Regex removendo markdown, mais lista branca de tipos de entidade

3. **UnicodeEncodeError derrubava o programa ao imprimir texto real**

   reconfigure(encoding="utf-8", errors="replace") no config

4. **A ordem das alternativas da regex apagava o rótulo específico**

   Ordenar os termos do mais longo para o mais curto

O erro 1 não gerou exceção nenhuma

O programa rodava, gerava gráfico e dava a resposta errada.

Só apareceu porque o bigrama mais frequente de todo o corpus era workflow failure, que é robô de CI e não pessoa discutindo IA.

Sentimento médio antes e depois do filtro de robô

Claude	GitHub Copilot	Gemini
−0,05 → +0,15	−0,10 → +0,12	−0,35 → +0,32

Demonstração

python main.py: menu com 10 opções, corpus em cache, roda sem internet

3

Regex em texto real

Antes e depois numa issue do corpus, com os padrões extraídos

5

spaCy token a token

Cada palavra com lema, classe gramatical e dependência

9

Busca KWIC

Padrão hallucinat\w+ mostrando como se fala de alucinação

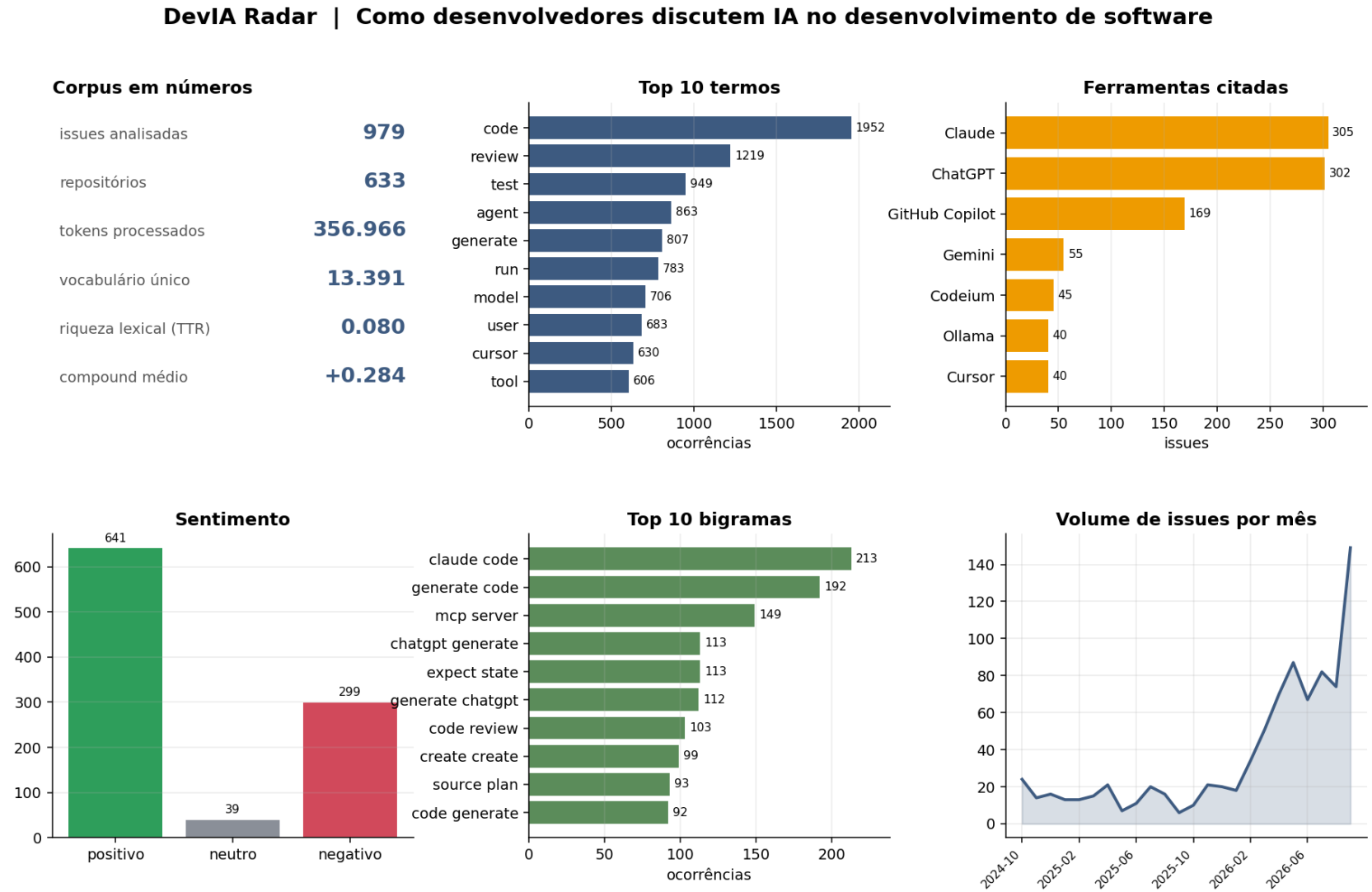
8

Dashboard

Os 8 gráficos gerados na pasta saida/

```
DevIA Radar - Etapa 1 - UFPB
estado: issues:1180 | corpus:979
---------------------------------
 [1] Coletar issues do GitHub
 [2] Ver resumo do corpus
 [3] Demonstrar as expressões regulares
 [4] Limpar e extrair padrões com regex
 [5] Demonstrar o spaCy token a token
 [6] Analisar o corpus com spaCy
 [7] Analisar sentimento com VADER
 [8] Gerar gráficos e dashboard
 [9] Buscar padrão no corpus (KWIC)
 [0] Rodar o pipeline completo
 [r] Recoletar do zero
 [s] Sair
```

Resultados



Volume cresceu 11x

De 13 issues em janeiro de 2025 para 149 em setembro de 2026, mês ainda incompleto.

Tom geral positivo

65,5% positivas contra 30,5% negativas, compound médio +0,28.

Revisão é o atrito

review é o 2º lema mais frequente de todo o corpus.

Ressalva: parte da curva de crescimento é viés da API, que favorece conteúdo recente.

Limitações assumidas

en_core_web_sm foi treinado em notícia e blog

Erra em texto técnico: marca Cursor e AI como país.

VADER não entende jargão

Lê critical e fatal como carga negativa, quando em issue são termos neutros.

A Search API limita a 1.000 resultados por consulta

O corpus é amostra, não população.

Próximas etapas

2

Áudio

Podcasts e entrevistas sobre IA no desenvolvimento. Duração, características acústicas e transcrição.

3

Vídeo

Palestras e vídeos técnicos. Frames, características visuais e transcrição.

→

Consolidação

O tema continua o mesmo nas três etapas, e o dashboard junta as três modalidades no fim.